

Trade-Off Log

Introduction

Perfection is a myth, especially when building something ambitious in just one week. True engineering is the art of strategic sacrifice: every choice is a deliberate trade-off where we consciously trade speed, simplicity, or cost for the features that matter most today. This log is not an admission of failure; it is a masterclass in prioritisation. It proves we have a surgical understanding of our architecture; we know exactly where our seams are, why we chose to leave them, and how we plan to reinforce them when we scale.

In the current Reflex build, we have focused our resources on three critical trade-offs: manual rider assignment, status-based tracking over live GPS, and our approach to simultaneous dispatcher actions.

Each of these represents a calculated decision to prioritise velocity without compromising the core integrity of the system. Below, we break down these decisions by identifying the specific weakness, justifying our current stance, and mapping out our future roadmap.

1. Manual rider assignment instead of automatic

Currently, our dispatch process relies on a manual selection system: when an order arrives, a dispatcher must personally scan a list and hand-pick a rider. Unlike modern, automated logistics, our current workflow lacks the intelligence to identify the closest, most available, or best-positioned courier for the job.

Why this is a critical bottleneck:

As our order volume and fleet size scale, this manual reliance becomes a significant liability. It introduces inevitable human error and paralysing delays. Dispatchers are forced to make high-stakes decisions under pressure without full visibility, frequently leading to sub-optimal assignments, such as tasking a rider who is miles away while a more efficient option remains overlooked. In a competitive delivery landscape, this lack of speed and precision is unsustainable.

Why we accepted it:

We originally adopted this manual approach because true automation demands sophisticated, real-time GPS tracking and a robust, high-performance matching algorithm. While building a system that instantly analyses rider location, workload, and efficiency is a complex engineering challenge, it is a necessary investment. Transitioning to automated dispatch is the single most

impactful step we can take to eliminate operational friction and unlock the next level of our business growth.

To build this feature correctly and ensure it operates with precision, avoiding errors like assigning a rider miles away, we require sufficient time for rigorous testing. Rushing this into a one-week build would force us to sacrifice the stability of the entire system just to favour a single component. By opting for a streamlined, manual assignment process, we have already delivered a transformative upgrade over the total lack of oversight found in WhatsApp's manual tracking.

What we'd do with more time

With additional development time, we will implement an intelligent "suggested rider" feature. This enhancement will leverage real-time location data to recommend the closest available driver, effectively eliminating guesswork while keeping the dispatcher firmly in control. This is the strategic approach to building a system that is both highly efficient and reliably human-centric.

2. Status-based tracking instead of live GPS

What's actually happening: Reflex tracks a delivery through four fixed stages: Assigned, Picked Up, Delivered (and Failed if something goes wrong). It does not show a live moving dot on a map following the rider's exact location in real time.

Why this is a real weak point: A retailer or customer might want to know exactly where the rider is *right now*, like watching an Uber car move on a map. Our system can't do that. It can only tell you the last stage the rider marked.

Why we accepted it: Live GPS tracking means the rider's phone has to constantly send its location to the server, usually every few seconds, even while the app is running in the background. This uses a lot of mobile data and drains the phone's battery fast, and many riders in this market use cheaper Android phones with smaller batteries and limited data plans.

Forcing that cost onto riders for a "nice to have" feature didn't seem like the right trade, especially when the core problem we're solving, "who is this delivery assigned to, and what stage is it at?", doesn't actually require live GPS to solve.

What we'd do with more time: Add optional location sharing that only turns on during an active delivery, and only shares location with the specific dispatcher and retailer for that one order, not constant background tracking. This gets most of the benefit without the battery and data cost.

3. No conflict handling for simultaneous dispatcher actions

What's actually happening: Imagine two dispatchers are both looking at the same list of riders at the same time. If both of them try to assign the *same* rider to two different orders in the same second, the system doesn't prevent it. Whichever assignment gets saved to the database last just quietly overwrites the first one, with no warning, no error message, nothing.

Why this is a real weak point: This is called a "race condition", a bug that only shows up when two things happen at almost the same moment. It's dangerous precisely because it's invisible: no one gets an error, so no one notices until a rider shows up confused about which delivery they're actually supposed to be doing, or a retailer wonders why their order shows no rider assigned even though someone clicked "assign."

Why we accepted it: In our current pilot use case, there's usually only one active dispatcher on shift at a time, so two people clicking "assign" on the same rider in the same instant is very unlikely; it's a rare edge case, not a daily risk. Building proper protection against this (called "locking") adds real complexity to the code, and we judged that fixing rare, low-probability bugs was less valuable this week than making sure the core flow- log assign, update status worked reliably for the common case.

What we'd do with more time: Add a database lock during the assignment step. In simple terms, when one dispatcher starts assigning a rider, the system "reserves" that rider's record for a split second so no one else can assign them until the first assignment finishes. In technical terms, this is a SELECT FOR UPDATE lock in Postgres; it forces simultaneous requests to happen one after another instead of overwriting each other.